Solution: BST Inorder Traversal

Problem Statement

Write Recursive Approach for Inorder Traversal of Binary Tree.

Given a binary tree, write a recursive algorithm to perform an inorder traversal of the tree and return the elements in the order they were visited.

Examples

Example 1:

```
Input:
           5
         /   \
        3     8
       / \   / \
      2   4 7   9

Output:
Inorder Traversal: [2, 3, 4, 5, 7, 8, 9]

Explanation:
The binary tree has the following structure:
           5
         /   \
        3     8
       / \   / \
      2   4 7   9

Performing an inorder traversal visits the nodes in ascending order: 2, 3, 4, 5, 7, 8,
9.
```

Example 2:

```
Input:
           10
         /    \
        6      15
       / \    / \
      3   8  12  18
            \
             9

Output:
Inorder Traversal: [3, 6, 8, 9, 10, 12, 15, 18]

Explanation:
The binary tree has the following structure:
           10
         /    \
        6      15
       / \    / \
      3   8  12  18
            \
             9

Performing an inorder traversal visits the nodes in ascending order: 3, 6, 8, 9, 10, 12,
15, 18.
```

Example 3:

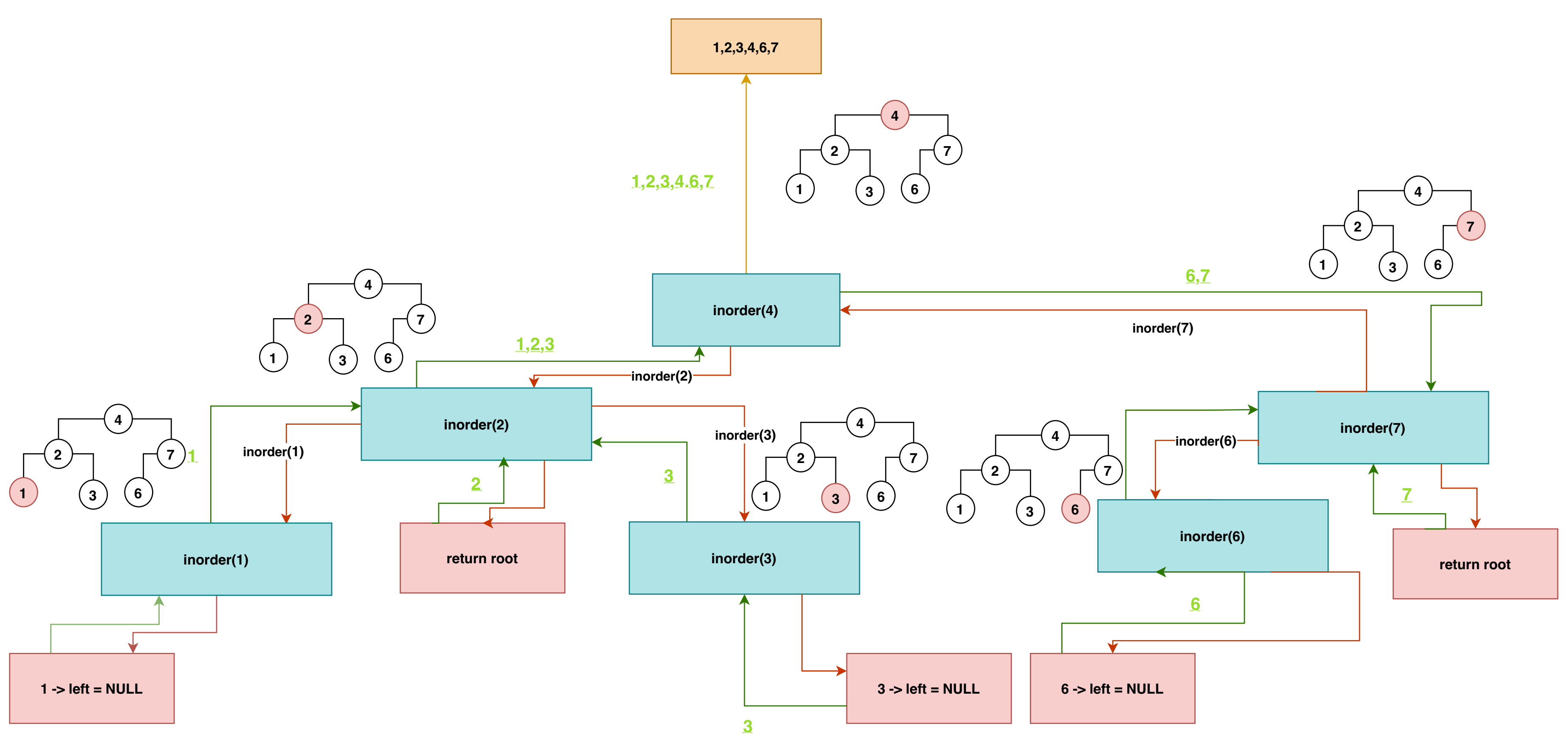
```
Input:
           20
         /    \
        12     25
       / \    / \
      8   15  22  28
     / \       \
    6   10      24

Output:
Inorder Traversal: [6, 8, 10, 12, 15, 20, 22, 24, 25, 28]

Explanation:
The binary tree has the following structure:
           20
         /    \
        12     25
       / \    / \
      8   15  22  28
     / \       \
    6   10      24

Performing an inorder traversal visits the nodes in ascending order: 6, 8, 10, 12, 15,
20, 22, 24, 25, 28.
```

Solution

The inorder traversal of a binary tree follows a recursive approach. We can define the algorithm as follows:

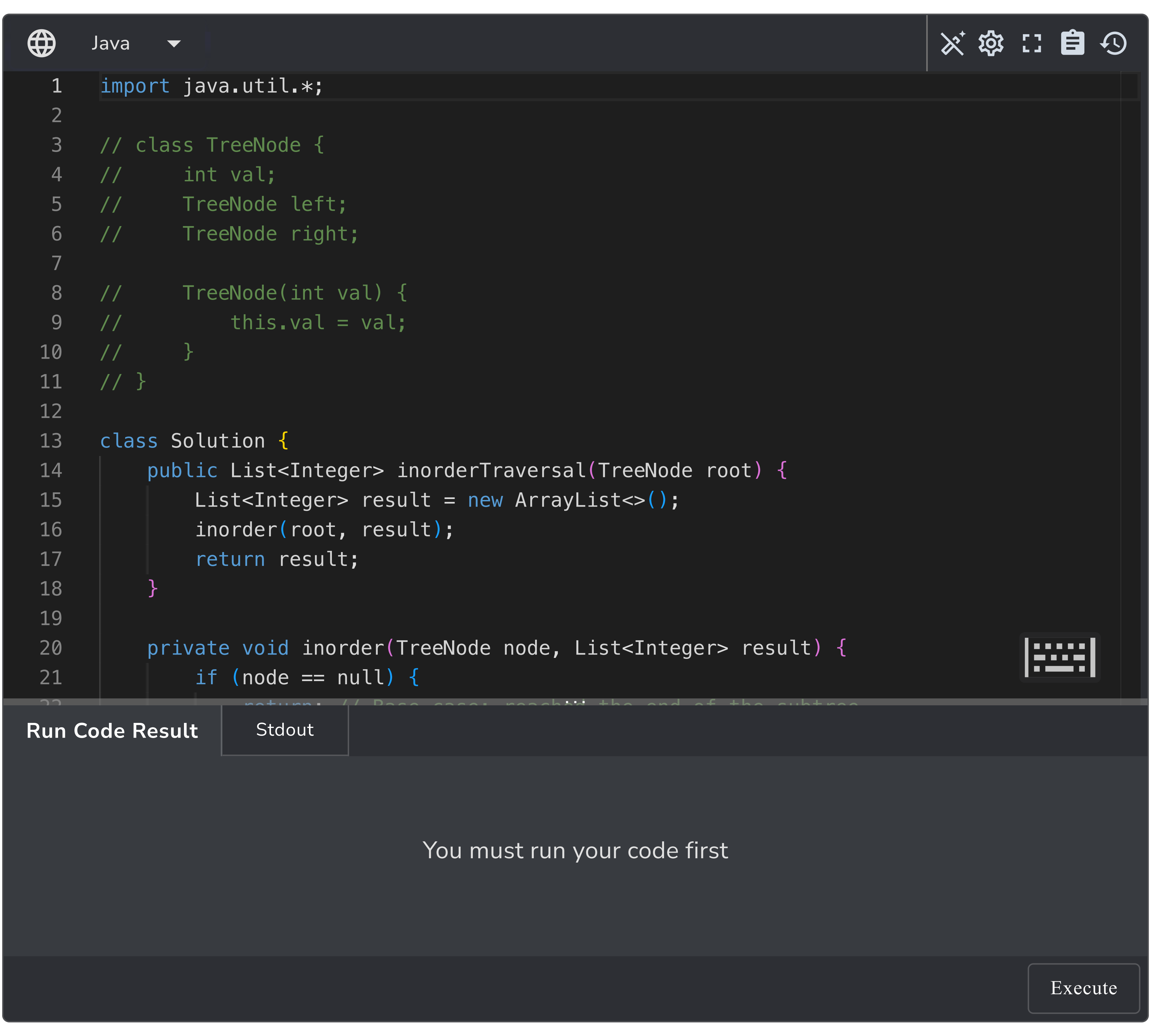
1. Base Case: If the current node is null, return.
2. Recursive Case:
   - Recursively traverse the left subtree.
   - Visit the current node.
   - Recursively traverse the right subtree.

By following this approach, we can perform an inorder traversal of the binary tree.



Code

Here is the code for this algorithm:



```java
import java.util.*;

// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;

//     TreeNode(int val) {
//         this.val = val;
//     }
// }

class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        inorder(root, result);
        return result;
    }

    private void inorder(TreeNode node, List<Integer> result) {
        if (node == null) {
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(N)$, where n is the number of nodes in the binary tree. This is because we need to visit each node exactly once.

The space complexity of the algorithm is $O(h)$, where h is the height of the binary tree. In the worst case, where the binary tree is skewed (i.e., a linked list), the height h is equal to the number of nodes, resulting in $O(N)$ space complexity. However, in a balanced binary tree, the height h is logarithmic to the number of nodes, resulting in efficient space usage with $O(logN)$ space complexity.

← Previous
11. BST Inorder Traversal
Next →
12. Merge Sort
☐ Mark as Completed